

المحاضرة الخامسة: المكونات الأساسية لواجهات التطبيقات في Flutter



م. سليمان الشوصي

محاور المحاضرة : contents

❖ ١. الهيكل الأساسي لتطبيقات Flutter

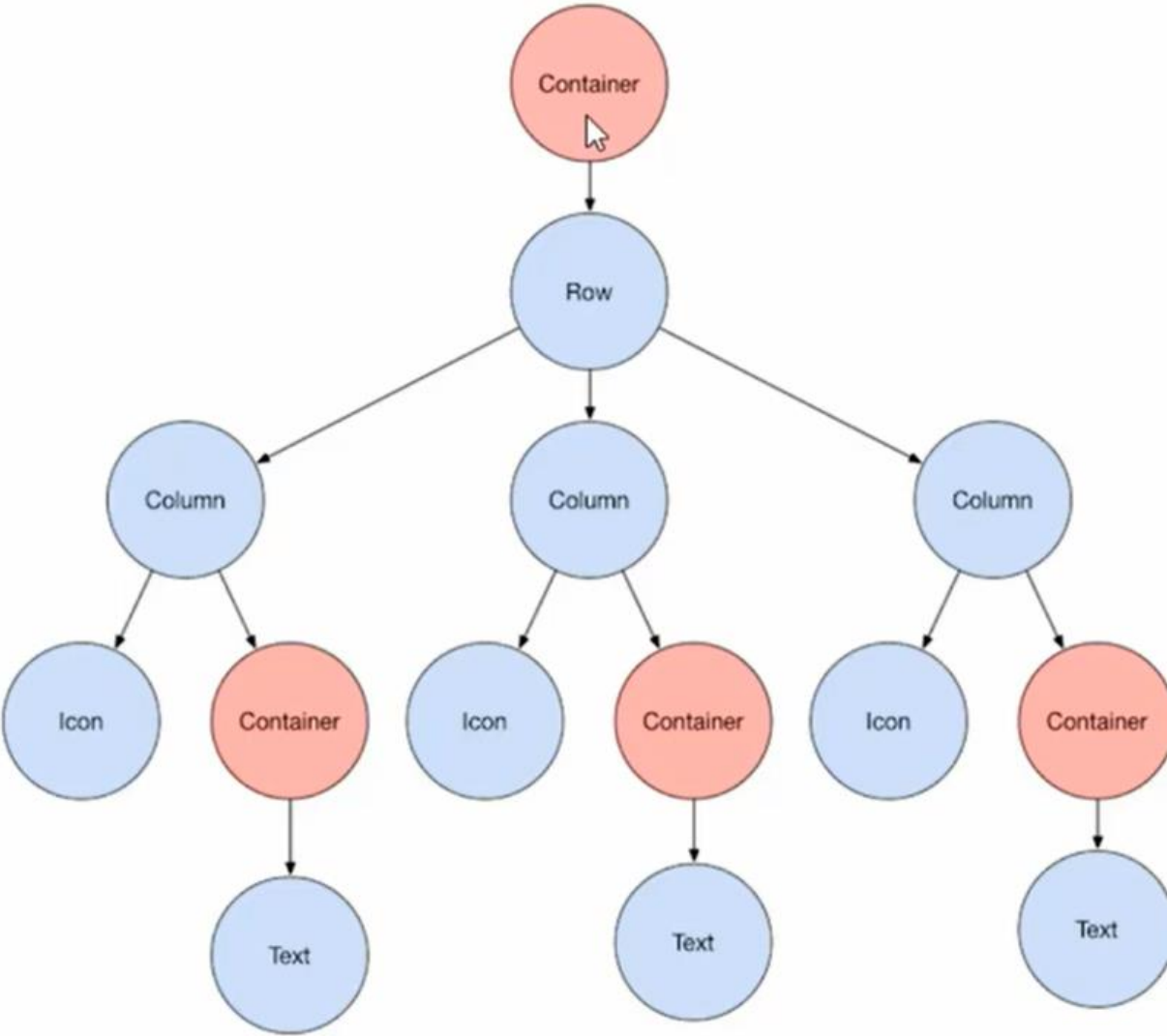
- فهم دالة البداية `main()` ودور `runApp()`.
- الفرق بين الواجهات الثابتة (`StatelessWidget`) والتفاعلية (`StatefulWidget`).
- دور `MaterialApp` والهيكل العام للشاشة `Scaffold`.
- مفاهيم السلامة وأهمية الكلمة المفتاحية `required`.

٢. بعض الأدوات الأساسية لبناء الواجهات (Core Widgets)

- عرض النصوص: الاستخدام المتقدم لأداة `Text` وتنسيق الخطوط (`TextStyle`).
- التصميم والحاويات: تفاصيل أداة `Container` والتشكيل الجمالي بـ `BoxDecoration`.
- الأيقونات والتفاعل: إضافة الرموز والأزرار التفاعلية (`Icon & IconButton`).

هيكل التطبيق التلقائي بداية انشاء المشروع في Flutter:

- **main():** نقطة البداية الأساسية لأي تطبيق Dart، حيث تستدعي الدالة `runApp()` لتشغيل وإطلاق التطبيق في الذاكرة.
- **runApp():** الدالة المسؤولة عن حقن وعرض الـ `Widget` الجذرية (Root Widget) على شاشة الجهاز.
- **StatelessWidget:** صنف واجهات ثابت لا يتغير حاله داخلياً (Stateless)، ويُستخدم لتصميم الهياكل التي لا تحتاج لتحديث بياناتها بصرياً.
- **StatefulWidget:** صنف واجهات تفاعلي متغير الحالة، ويمتلك كبيراً مستقلاً لإدارة الحالة (State) لتحديث الواجهة لحظياً عند تغير البيانات (مثل عداد الـ `counter`).
- **MaterialApp:** هي widget الأساسي الذي يضبط إطار التصميم العام للتطبيق (Material Design)، ويتحكم بالثيمات، والعناوين، وصفحات التنقل (Navigation). لكي يتم بناء التطبيق.
- **Scaffold:** الهيكل العظمي للشاشة، ويوفر الإطار الجاهز لعناصر واجهة المستخدم مثل شريط العلوي (AppBar)، جسم الشاشة (body)، والزر العائم (floatingActionButton).
- **كلمة required:** تُستخدم في معاملات الدوال أو المُنشئات (Constructors) للإشارة إلى أن هذا المعامل إلزامي ولا يمكن تجاوزه، ولن يقبل المترجم تشغيل الكود إن لم تقم بتمرير قيمة له.



Widget Tree

مكتبة `import 'package:flutter/material.dart';` هي عبارة عن قطعة تحتوي على مجموعة من القطع الأدوات (widgets)

تنقسم الأدوات إلى مجموعات حسب دورها، وهي أساسيات بناء واجهات المستخدم (UI) في flutter:

المجموعة الأولى: الأساسيات والمواد (Core & Material) : هي الـ Widgets الأساسية المطلوبة لإنشاء أي تطبيق بـ فلاتر:

١. الدالة الأساسية `runApp ()` ليست Widget ، بل هي الدالة الرئيسية التي يبدأ بها تنفيذ التطبيق في فلاتر، هي نقطة الدخول (Entry Point) للتطبيق، حيث تستقبل widget واحد فقط وهو الـ widget الجذر Root Widget الذي يمثل التطبيق بأكمله.

٢. الحاوية الشاملة `MaterialApp` : هذا الـ Widget يحيط بالتطبيق ويوفر له سياق (Context) وخدمات خاصة بتصميم Material Design (تصميم جوجل). يتم استخدامه مرة واحدة كـ Widget جذر للتطبيق، ومن أبرز خصائصه كما هو موضح في الجدول التالي.

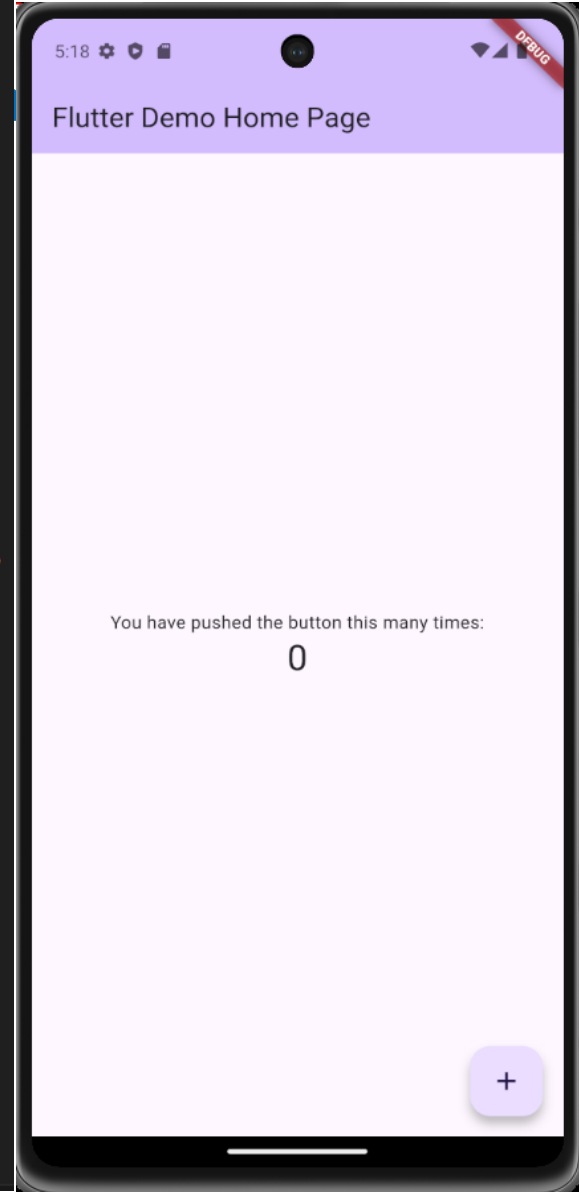
الوصف.	خصائص MaterialApp
الأكثر أهمية. تحدد الـ Widget الذي سيتم عرضه كـ الشاشة الرئيسية الافتتاحية للتطبيق.	home (Widget)
النص الذي يظهر في مدير المهام (Recent Apps) في نظام التشغيل (وليس عنوان الشريط العلوي).	title (String)
لتحديد المظهر العام للتطبيق (الألوان الافتراضية، الخطوط، شكل الأزرار).	theme (ThemeData)
خاصية تحكمية. إذا كانت true (القيمة الافتراضية)، يظهر شريط "DEBUG" في الزاوية. يُفضل تعيينها إلى false في مرحلة الإنتاج.	debugShowCheckedModeBanner (bool)

٣. الهيكل الأساسي للشاشة Scaffold: هو ال-Widget الذي يوفر هيكل (Structure) مرئياً ومعياريّاً لشاشة واحدة في تصميم Material Design. يوفر أماكن محددة للعناصر القياسية مثل شريط العنوان والجسم الأساسي، من أهم المكونات الأساسية الموضحة في الجدول التالي:

أبرز المكونات	الوصف
AppBar (PreferredSizeWidget)	شريط العنوان العلوي الذي يظهر في أعلى الشاشة، ويُستخدم عادةً AppBar Widget.
body (Widget)	المحتوى الرئيسي الذي يشغل معظم مساحة الشاشة، ويقع بين appBar و bottomNavigationBar.
floatingActionButton (Widget)	زر الإجراء العائم (FAB) ، يظهر عادةً في الزاوية السفلية اليمنى ويستخدم لإجراء رئيسي واحد في الشاشة.
bottomNavigationBar (Widget)	شريط التنقل الذي يظهر في أسفل الشاشة، ويستخدم عادةً BottomNavigationBar Widget للتنقل بين شاشات رئيسية.
Drawer (Widget)	القائمة الجانبية (Sidebar) التي تنزلق من جانب الشاشة (عادةً اليسار).

```
1 import 'package:flutter/material.dart';
  Run | Debug | Profile
2 void main() { runApp(const MyApp());
3 }
4 class MyApp extends StatelessWidget {
5   const MyApp({super.key});
6   @override
7   Widget build(BuildContext context) {
8     return MaterialApp( title: 'Flutter Demo'
9       theme: ThemeData(colorScheme:
10         ColorScheme.fromSeed(
11           seedColor: Colors.deepPurple),), //
12       home: const MyHomePage(
13         title: 'Flutter Demo Home Page'), //
14     );} // MaterialApp
15 }
16 class MyHomePage extends StatefulWidget {
17   const MyHomePage(
18     {super.key, required this.title});
19   final String title;
20   @override
21   State<MyHomePage>
22   createState()=>_MyHomePageState();}
```

```
23 class _MyHomePageState extends State<MyHomePage>{
24   int _counter = 0;
25   void _incrementCounter(){
26     setState((){_counter++;}); }
27   @override
28   Widget build(BuildContext context){
29     return Scaffold(appBar: AppBar(backgroundColor:
30       Theme.of(context).colorScheme.inversePrimary,
31       title: Text(widget.title),), // AppBar
32     body: Center(child: Column(
33       mainAxisAlignment: MainAxisAlignment.center,
34       children: <Widget>[
35         Text('You have pushed the button this many times:'),
36         Text('$_counter',
37           style:Theme.of(context).textTheme.headlineMedium,
38         ),]), // Text // <Widget>[] // Column // Center
39     floatingActionButton:FloatingActionButton(
40       onPressed:_incrementCounter,
41       tooltip: 'Increment',
42       child: const Icon(Icons.add),
43     ));} // FloatingActionButton // Scaffold
44   }
45 }
```



المجموعة الثانية: الـ Widgets التي لا تحتوي على حالة (Stateless Widgets):

الـ Stateless Widgets هي عناصر واجهة المستخدم التي لا تتغير أثناء تشغيل التطبيق. بمجرد بنائها، تظل كما هي. يتم استخدامها لعرض المعلومات الثابتة أو التي تأتي من خصائص يتم تمريرها إليها، ومن بعض هذه العناصر التالية:

١. الفئة الأساسية **StatelessWidget()**: هو القالب الذي يتم استخدامه لبناء جميع الـ Widgets الثابتة، لا تحتوي على حالة داخلية تتغير. بل هي مجرد دالة **build** (**build (BuildContext context)**) تستقبل خصائص وتبني واجهة المستخدم، ومن أهم خصائصها أو مكوناتها هي الدالة **build (BuildContext context)**: وهي الدالة الوحيدة الضرورية التي يجب تجاوزها، وظيفتها هي ارجاع الـ widget الذي سيعرض على الشاشة.

```
47 class MyStaticCard extends StatelessWidget {  
48   // يمكن تمرير بيانات ثابتة عبر المُنشئ (Constructor)  
49   final String title;  
50   const MyStaticCard({super.key, required this.title});  
51   @override  
52   Widget build(BuildContext context) {  
53     // الذي يمثل هذه البطاقة Widget ترجع الـ  
54     return Card(  
55       child: Center(  
56         child: Text(title, style: const TextStyle(fontSize: 18)),  
57       ), // Center  
58     ); // Card  
59   }  
60 }
```

٢. أداة **Text** (عرض النصوص): هي الأداة لعرض سلسلة من الأحرف أو النصوص وتنسيق خطوطها وألوانها على الشاشة، وهو أحد أكثر الـ Widgets استخداماً.

أهم الخصائص:

- **string**(النص المباشر): النص المراد عرضه.
- **Style**: يُحدد تنسيقات الخط عبر كائن **TextStyle**(مثل اللون، الحجم، وسمك الخط).
- **textAlign**: يُحدد محاذاة النص داخل الإطار (يمين، يسار، وسط، أو ضبط الكتل justify).
- **maxLines**: يُحدد أقصى عدد للأسطر المسموح لعرض النص فيها.
- **Overflow**: يُحدد كيفية التصرف إذا زاد النص عن المساحة المتاحة (مثل وضع نقاط TextOverflow.ellipsis).

```
Text('!فلاتر: إطار عمل متقاطع المنصات',  
  textAlign: TextAlign.justify,  
  style: const TextStyle(  
    color: Color(0xFF1E88E5), // لون أزرق  
    fontSize: 18.0,  
    fontStyle: FontStyle.italic,  
  ), // TextStyle  
  maxLines: 1,  
  overflow: TextOverflow.ellipsis,  
  // إضافة نقاط (...) إذا تجاوز النص السطر  
) // Text
```

٢. أداة `Icon` أو `IconButton` (الأيقونات والأزرار): `Icon` لعرض رموز بصرية ثابتة، و `IconButton` لتقديم أيقونة قابلة للضغط كزر تفاعلي.

أهم الخصائص:

- **Icon**: رمز الأيقونة المطلوب من مكتبة `Icons` الجاهزة.
- **Size**: حجم الأيقونة بالبكسل.
- **Color**: لون الأيقونة.

```
const Icon(Icons.school,  
  size: 50.0,  
  color: Colors.deepPurple),
```

المجموعة الثالثة: الـ Widgets التي تحتوي على حالة (Stateful Widgets): الـ StatefulWidgets هي عناصر واجهة المستخدم التي يمكن أن تتغير (تتفاعل) استجابة لأحداث المستخدم (مثل الضغط على زر) أو لتغير البيانات. على عكس StatelessWidget، تحتاج هذه الـ Widgets إلى إدارة حالة داخلية:

١. **Stateful Widget (الفئة المنشئة):** هي مجرد فئة مُنشئة لا تتغير بحد ذاتها، بل وظيفتها الأساسية هي إنشاء وإدارة كائن **State** المرافق لها. يحتوي على دالة واحدة رئيسية تُنشئ الحالة وهي الدالة **createState()** الدالة الوحيدة الضرورية. وظيفتها هي إرجاع مثيل من كائن **State** الذي سيحتوي على الحالة والواجهة المتغيرة.

٢. **State (الفئة المُتحكّمة في الحالة):** كائن **State** هو الجزء الديناميكي الذي يدير البيانات المتغيرة والواجهة المرتبطة بها. يُعاش هذا الكائن طالما أن الـ **Widget** موجود في شجرة العناصر.

المكونات: State<T>: الفئة الأساسية التي تحتوي على الحالة (البيانات) ودورة حياة الـ **Widget**، يتم تعريف البيانات المتغيرة كمتغيرات داخل هذه الفئة. ومن خصائصها:

- **build (BuildContext context):** مثل StatelessWidget، لكنها تعتمد على البيانات الداخلية.
- **initState():** تستدعى مرة واحدة عند إنشاء الكائن. مثالية لتهيئة البيانات الأولية.
- **dispose():** تستدعى عند إزالة الـ **Widget** بشكل دائم. مثالية لتنظيف المصادر.

setState(void Function () fn): الدالة الأهم: تستخدم لإخبار التطبيق بأن الحالة (البيانات) قد تغيرت ويجب إعادة بناء الـ **Widget** إعادة استدعاء دالة **(build())** لعرض التغير. يجب وضع أي تغيير في متغيرات الحالة داخل هذه الدالة. حيث **fn:** دالة بدون مدخلات تقوم بتغيير متغيرات الحالة.

أهمية setState():

- **من أحد الأخطاء الشائعة:** عندما نقوم بتغيير قيمة **_counter** او قيمة أي مكونات أخرى مباشرة دون استدعاء **setState()**، ستتغير البيانات فعلاً، لكن في واجهة التطبيقات في فلاتر لن يعلم بذلك، ولن يعيد بناء الواجهة، وستظل الشاشة تعرض القيمة القديمة.
- **الآلية الصحيحة** **setState()**: تخبر نظام فلاتر: "هناك تغيير في البيانات، أعد تشغيل دالة **(build())** لهذه الـ **Widget** حتى تعكس الواجهة القيمة الجديدة."

```

21 // 1. فئة StatefulWidget (ثابتة)
22 class MyHomePage extends StatefulWidget {
23   // لا تغير الخصائص هنا
24   const MyHomePage({super.key, required this.title});
25   final String title;
26   @override
27   State<MyHomePage> createState() => _MyHomePageState(); // استدعاء الحالة
28 }
29 // 2. فئة State (ديناميكية)
30 class _MyHomePageState extends State<MyHomePage> {
31   // 1. تعريف الحالة (البيانات المتغيرة)
32   int _counter = 0;
33   void _incrementCounter() {
34     setState(() {
35       _counter++;
36     });
37   }
38   @override
39   Widget build(BuildContext context) {
40     return Scaffold(
41       appBar: AppBar( // AppBar ...
42         body: Center(
43           child: Column(
44             mainAxisAlignment: MainAxisAlignment.center,
45             children: <Widget>[
46               const Text('قيمة العداد الحالية هي'),
47               Text('$ _counter',
48                 style: const TextStyle(fontSize: 48, fontWeight: FontWeight.bold),
49               ), // Text
50               ElevatedButton( onPressed: _incrementCounter, // استدعاء الدالة
51                 child: const Icon(Icons.add),), // ElevatedButton
52             ], // <Widget>[]
53           ), // Column
54         ), // Column
55       ), // Scaffold
56     );
57   }
58 }

```

```

Container(
  height: 120,
  width: double.infinity,
  margin: const EdgeInsets.symmetric(
    horizontal: 20, vertical: 15), // EdgeInsets.symme
  padding: const EdgeInsets.all(10),
  decoration: BoxDecoration(
    color: Colors.lightBlue[100], // لون الخلفية
    borderRadius: const BorderRadius.only(
      topLeft: Radius.circular(30),
      bottomRight: Radius.circular(30),
      topRight: Radius.circular(5),
      bottomLeft: Radius.circular(5)), // BorderRadius.o
    // (Border) والحد (Box Shadow) خصائص إضافية: الظل
    border: Border.all(
      color: Colors.blue.shade700,
      width: 2.5, // Border.all // Border.all
    ),
    boxShadow: [
      BoxShadow(color: Colors.black.withOpacity(0.3),
        spreadRadius: 2, // مدى انتشار الظل
        blurRadius: 8, // مدى نعومة (غبش) الظل
        offset: const Offset(4, 4), // إزاحة الظل (x, y)
      ), // BoxShadow // BoxShadow
    ], // BoxDecoration // BoxDecoration
  ), // Container
  child: Text("this is container widget"), // Container

```

❖ **المجموعة الرابعة: ال Widgets الهيكلية والتنظيم (Layout Widgets)** تستخدم هذه ال Widgets لتحديد موقع وأبعاد ال Widgets الأخرى على الشاشة. في فلاتر يعتمد على نظام "القيود والحرية (Constraints and Liberties)، حيث يتم تحديد مكان كل Widget داخل ال widget الرئيسي له، ومن بعض مثله هذه ال widget هي التالية:

١. أداة **Container (الحاوية الشاملة):** أداة متعددة الأغراض تُستخدم كصندوق مرّن يمكنه دمج (جمع) بين خصائص الأبعاد، الهوامش، الحواشي، والتصميم واللون.

أهم الخصائص:

- **width / height:** لتحديد عرض وارتفاع الحاوية، القيمة **double.infinity** يجعل ال Container يمتد ليأخذ كامل عرض المساحة المتاحة (عرض الشاشة).
- **Child:** العنصر الوحيد الذي يوضع داخل الحاوية Container.
- **Margin:** المساحة أو المسافة الفاصلة خارج حدود الحاوية عن العناصر المجاورة له، **EdgeInsets.symmetric** لإضافة هامش على الجانبين (أفقياً) وعمودياً من الأعلى والأسفل.
- **Padding:** المساحة أو المسافة الفاصلة داخل الحاوية بين حدودها والعناصر التي بداخلها.
- **Decoration (BoxDecoration):** تأخذ كائن BoxDecoration للتحكم في لون الخلفية، حواف الإطار، والظلال.

❖ **أداة BoxDecoration (تنسيق الحاويات)** هو كائن مُساعد يُستخدم داخل خاصية decoration في ال Container لتطبيق مؤشرات بصرية تقدمية.

- **Color:** لون خلفية الصندوق.
- **border: Border.all** رسم حدود حول الصندوق مع تحديد اللون والسمك.
- **borderRadius: BorderRadius.only()** هذه الخاصية التي تركز على تحديد استدارة كل زاوية على حدة، مما يعطي شكلاً غير متماثل: **topLeft** و **bottomRight** استدارة كبيرة (٣٠). **topRight** و **bottomLeft** استدارة صغيرة بمقدار 5 بكسل.
- **boxShadow:** تأثير الظل الذي يعطي عمقاً ثلاثي الأبعاد للصندوق: **color** لون الظل مع شفافية (%). **blurRadius** يجعل الظل ناعماً وغير حاد: **offset** يحرك الظل لأسفل ولليمين ليعطي إحساساً بالإضاءة من الأعلى واليسار.

Thank you